Course : AAD – ALGORITHM ANALYSIS AND DESIGN

Lecture 7

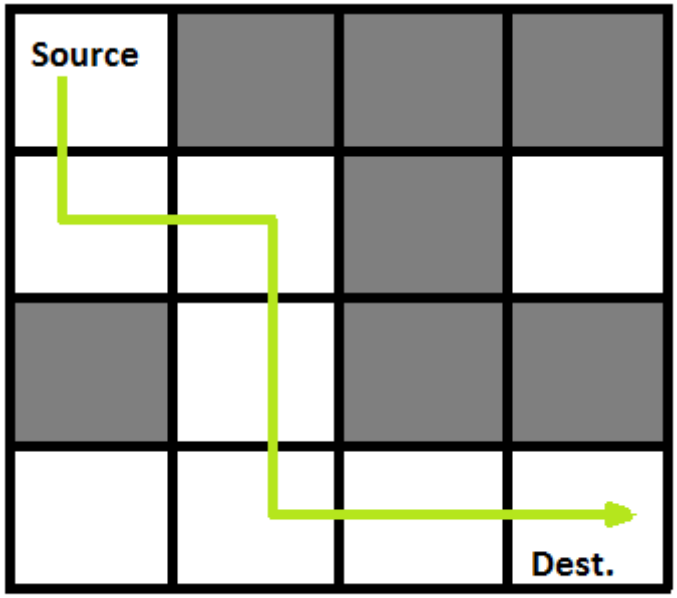
Design and analysis of Backtracking Algorithms - 2

Dr Rahaf Ghazal

2025-2026

Example 3: Finding a path through a maze:

Path in a Grid or Maze refers to problems that involve passing through a grid-like structure from the source (starting point) to the goal (endpoint) while avoiding the blocks.



Path in a Maze/Grid

Grid and Maze Representation:

A grid or a maze is generally represented as a 2D array or matrix consisting of rows and columns. The cells in the grid may be open for traversal or may be blocked by an block(this cell cannot be visited).

Common Pathfinding Problems:

Finding a Path: The most standard problem involves finding path from a given starting point to a given ending point in the grid, without visiting any block. Usually we use the *Backtracking algorithm* (is used when exploring all possible paths from source to destination) for solution.

Counting Paths: In some of the problems we have find the total number of unique paths from the starting point to the ending point under some rules and constraints. Usually we use the *Dynamic algorithm* (to avoid redundant calculations in Maze Problems) for solution

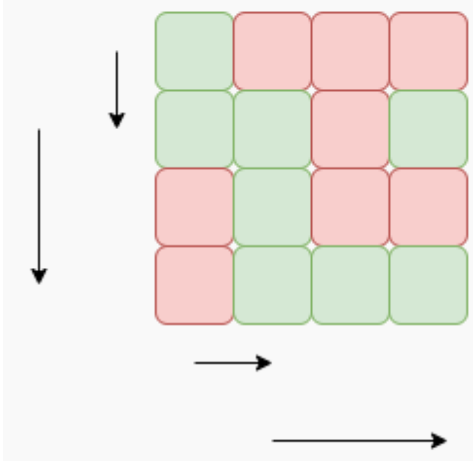
Shortest Path: In some of the problems, the task will be to find the shortest path from the source to the destination. Usually we use the *Greedy algorithm (Dijkstra's algorithm)* for solution.

Rat in a Maze Problem

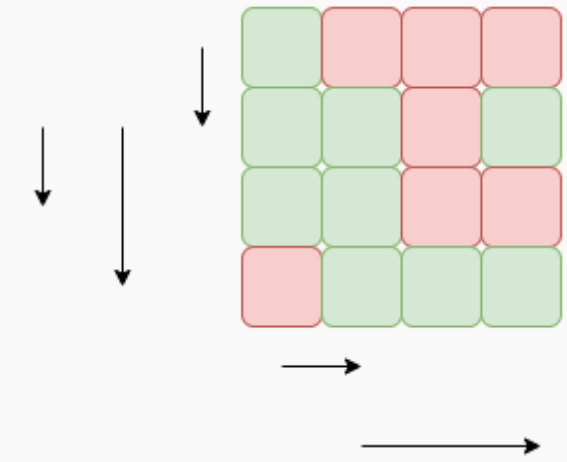
Consider a rat placed at **(0, 0)** in a square matrix **m[ ][ ]** of order n and has to reach the destination at **(n-1, n-1).** The task is to find a sorted array of strings denoting all the possible directions which the rat can take to reach the destination at (n-1, n-1). The directions in which the rat can move are **'U'(up), 'D'(down), 'L' (left), 'R' (right).**

Examples:

Input : *N = 4*
1 0 0 0
1 1 0 1
0 1 0 0
0 1 1 1
Output :
DRDDRR



Input : *N = 4*
1 0 0 0
1 1 0 1
1 1 0 0
0 1 1 1
Output :
DDRDRR DRDDRR



Backtracking Algorithm

Input: A maze represented as a 2D array, where 1 represents an open space and 0 represents a wall.

Algorithm:

1. Start at the starting point.
2. For each of the four possible directions (up, down, left, right), try moving in that direction.
3. If moving in that direction leads to the ending point, return the path taken.
4. If moving in that direction does not lead to the ending point, backtrack to the previous position and try a different direction.
5. Repeat steps 2-4 until the ending point is reached or all possible paths have been explored.

Try coding ^_^

```
// function:   getallpath

// Inputs:          maze (N x N grid, 1=open, 0=blocked), n (size)

// Output:        List of all valid paths (strings like "DDRR")
```

//نعرف الدالة التراجعية العودية التي سوف تقوم بأخذ وسطائها بالشكل: مصفوفة المتاهة وحجمها والموقع الحالي، بالإضافة لمصوفة متجهات سوف تحمل جميع الحلول المقترحة في نهاية الخوارزمية وأخيراً سلسلة الحل الذي سوف نبنيه بشكل تراكمي(باستخدام العودية)

```
void getallpath(int matrix[MAX][MAX], int n,int row,int col,vector<string>

&ans,string cur)

   {
```

1) فحص المعطيات المدخلة (هل الموقع الحالي مقبول بمعنى أننا مازلنا داخل المتاهة وكذلك هل هو مشار له بأنه مفتوح) فإذا انتقض أحد الشرطين يتم إنهاء الدالة لعدم قبول الحل

```
if(row>=n or col>=n or row<0 or col<0 or matrix[row][col] == 0)
    return ;
```

2) تعريف حالة النجاح وهي عندما يصل الفأر إلى آخر المتاهة ، عندها نضيف الطريق المسجل cur إلى مصفوفة الحلول ans

```
if(row == n-1 and col == n-1)
    {   ans.push_back(cur);     return ;   }
```

3)بعد اختبار صحة الانتقال نقوم بتسجيل الخطوة. حيث نقوم بإغلاق الطريق الذي ممرنا به بشكل مؤقت ومن ثم نجرب جميع الاتجاهات الممكنة للانتقال التالي:

```
matrix[row][col] = 0;
    getallpath(matrix,n,row-1,col,ans,cur+"U");
    getallpath(matrix,n,row,col+1,ans,cur+"R");
    getallpath(matrix,n,row,col-1,ans,cur+"L");
    getallpath(matrix,n,row+1,col,ans,cur+"D");
```

4) انتهاء تنفيذ الاستدعاءات الأربعة السابقة يعني تحقق الخطوة 1 أو 2، عندها نتراجع في الحل إلى خطوات سابقة وننهي التنفيذ للدالة الحالية إما للبحث عن حلول أخرى أو للبحث في اتجاهات صحيحة.

```
matrix[row][col] = 1;
    } // end function getallpath
```

Time Complexity of algorithm

For a standard ***nxn*** grid-based maze using a backtracking or Depth-First Search (DFS) approach to explore every possible path, the time complexity is exponential. In the worst case, every cell could lead to 4 recursive calls, resulting in a complexity of approximately **O(4^(n*n)).** This is because at each cell, the algorithm might explore up to four directions, and in the worst case, many paths are valid. (**try draw the solution tree**)

Example 4: Target Sum Combinations:

Given an array arr[] of distinct integers and an integer target, find all unique combinations of array where the sum of chosen element is equal to target. The same element may be chosen any number of times to make target.

For reading

Examples:

Input: *arr[] = [1, 2, 3], target = 5*
Output: *[[1, 1, 1, 1, 1], [1, 1, 1, 2], [1, 1, 3], [1, 2, 2], [2, 3]]*
Explanation: *All the combination have sum of elements equals to target.*

Input: *arr[] = [2, 4], target = 1*
Output: *[]*
Explanation: *No combination exits whose sum is equals to target*

The approach:

The idea is to explore all possible combinations of numbers that add up to the target. For each element, we reduce the target by its value and continue the process recursively. If at any point the target becomes zero, it means we have found a valid combination. On the other hand, if the target goes negative, we backtrack and discard that path.

Pseudocode idea

// function: FindCombinations

// Inputs: array of integer numbers, integer number(target sum)

// Output: List of all valid subsets which have sum equal to the target

```
Function FindCombinations(Candidates, Target, Index, CurrentCombination):
```

1. قاعدة التوقف (النجاح): إذا وصل المجموع للهدف تماماً

```
    If Target == 0:
    Print(CurrentCombination)        //    وجدنا تشكيلة صحيحة، نقوم بحفظها أو طباعتها
        Return
```

2. قاعدة التوقف (الفشل): إذا تجاوزنا الهدف أو انتهت الأرقام

```
If   Target < 0   OR    Index == Candidates.Length:
        Return
```

3. الخيارات المتاحة (الحلقة):

```
For i From Index To Candidates.Length - 1:
```

أ. إضافة العنصر الحالي للتشكيلة

```
 CurrentCombination.Add(Candidates[i])
```

ب. الاستدعاء العودي (البحث عن المتبقي من المجموع)

ملاحظة: نمرر i إذا كان مسموحاً تكرار نفس الرقم، أو i+1 إذا كان غير مسموح

```
FindCombinations(Candidates, Target - Candidates[i], i, CurrentCombination)
```

ج. التراجع إزالة العنصر لتجربة رقم آخر

```
CurrentCombination.RemoveLast()
```

Time Complexity of algorithm

O(n^(⌈T/M⌉)), where n is the number of candidates, T is the target, and M is the smallest candidate.

Backtracking can be visualized as a DFS on an n-array tree:

- Each node represents a choice of candidate, so it can have at most n children.
- The maximum depth is ⌈T/M⌉, because in the worst case we keep adding the smallest candidate until reaching the target.
- The total number of nodes in an n-array tree of depth T/M is bounded by n^(T/M), giving the loose upper bound.